

# Introduction à la programmation Python pour les sciences des données

Séance 1 : Le cursus Python, apprentissage machine et IA dans le parcours Mageval

---

Thomas Thivillon

8 septembre 2026

## À propos de ce cours

- À la fois un cours d'informatique et un cours de méthodes statistiques.
- Premier élément d'une série de 3 cours : Mageval 1, Mageval 2, Mageval 3.
- En Mageval 1 : 2 x 12h réparties sur 2 semestres.
- **Objectifs sur l'ensemble du cursus :**
  - Maîtriser la programmation Python (conception, diagnostic, debug) pour les sciences des données au sens large.
  - Bonne connaissance des méthodes statistiques prédictives en particulier, et capacité à les mettre en oeuvre avec Python.
  - Culture IA : compréhension générale du fonctionnement des grands modèles de langage (LLM).

- **Python :**

- Un langage de programmation sous licence libre (depuis 1991).
- Fort gain de popularité au cours de la décennie 2010.
- Très grande versatilité : IA et machine learning, développement web, développement d'applications, calcul scientifique, développement de jeux...
- S'utilise le plus souvent avec un environnement de développement intégré (IDE). Ex : Jupyter, VSCode.
- Utile dans toutes les phases du traitement de données : manipulation, structuration, modélisation.

- **Intelligence artificielle (IA) :**
  - AI Act UE : “Système basé sur une machine qui est conçu pour fonctionner avec différents niveaux d'autonomie et qui peut faire preuve d'adaptabilité après son déploiement, [...], déduit, à partir des données qu'il reçoit, comment générer des résultats tels que des prédictions, du contenu, des recommandations ou des décisions [...]”.
  - Recouvre :
    - IA réactive : tâches de prédiction pure sans mémoire (reconnaissance faciale, retranscription de texte manuscrit, etc).
    - IA à mémoire limitée : IA générative (texte, image, musique, code), assistants virtuels/chatbots, engins autonomes.
    - IA non-réalisée : IA à théorie de l'esprit et IA consciente.

- **Machine learning (ou apprentissage machine) :**
  - Champ de recherche issu des sciences statistiques et des sciences informatiques.
  - Vise à permettre à une machine d'effectuer des tâches de **prédiction** ou de **classification** suite à un processus d'**entraînement** sur un jeu de données.
  - Exemple d'application : prédire le niveau de vie d'un individu à partir de traces de son activité téléphonique (CDR : call detail records).
  - Forme d'IA réactive, beaucoup moins coûteuse à entraîner qu'un modèle d'IA générative mais avec de nombreuses applications extrêmement utiles.

## Machine learning (ML) et IA : quel rapport ?

- Les outils d'IA réactive s'appuient notamment sur les méthodes du machine learning, en particulier sur les modèles de réseaux de neurones.
- L'IA à mémoire limitée mobilise des modèles dérivés des réseaux de neurones mais qui sont plus sophistiqués (réseaux de neurones profonds, réseaux de neurones convolutifs).
- Etudier les méthodes de machine learning permet d'avoir une première compréhension du fonctionnement des modèles d'IA les plus avancés comme les grands modèles de langage utilisés par OpenAI (ChatGPT), Anthropic (Claude), etc.
- C'est aussi un passage obligé pour ceux qui veulent aller plus loin dans l'étude de ces modèles.

# À quoi bon apprendre à programmer à l'ère des IA génératives ?

- Les IA savent très bien coder mais elles le font en fonction de la description d'un problème et de son contexte que leur fournit un humain.
- Si ces éléments sont mal décrits, elles peuvent produire des solutions inappropriées ou erronées et ne pas le détecter.
- Nous avons besoin de penser comme un programmeur pour bien formuler les problèmes et bien décrire notre contexte, nos objectifs, nos contraintes.
- Nous avons aussi de bien comprendre le langage Python et sa syntaxe pour vérifier que le code généré répond à nos objectifs et tient compte des contraintes que nous avons formulées.

# Structuration des contenus sur l'ensemble du parcours

- **Année 1 :**

- Bases de la syntaxe Python (boucles, conditions, fonctions, calcul).
- Manipulation de données (fusionner, filtrer, trier, décrire).
- Premières applications liées à l'IA (descentes de gradients, natural language processing).

- **Année 2 :**

- Introduction au ML (méthodes d'apprentissage supervisé).
- Méthodes de régularisation (LASSO, ridge, elastic net).

- **Année 3 :**

- Arbres de régression, random forests, et gradient boosting.
- Introduction aux réseaux de neurones.

## Plan du cours pour ce semestre (8 séances)

1. Introduction générale
2. Prise en main de Python
3. Structures conditionnelles
4. Boucles et fonctions
5. Evaluation de mi-semestre
6. Calcul avec NumPy
7. Graphiques
8. Evaluation de fin de semestre

- Examen de mi-semester (40%).
- Examen de fin de semestre (50%).
- Participation (10%), basé sur :
  - Présence aux séances.
  - Exercices à la maison.
  - Contribution aux discussions en séance.

## Exercices à la maison :

- Non-notés sur le contenu mais leur rendu régulier compte dans la note de participation.
- Vous permettent d'acquérir des automatismes en programmation Python.
- Très difficile de réussir les examens si vous ne les faites pas chaque semaine.
- On ne devient pas bilingue dans un nouveau langage en s'y mettant la veille de l'examen...

- Service Jupyter-Python de SSP Cloud.
- Nombreux bénéfices :
  - Fonctionne sur tous les ordinateurs (y compris salles informatiques).
  - Version à jour de Python, pas de bug.
  - Compatible avec le contrôle de versions GitHub.
  - Capacités de calcul élevées (jusqu'à 50 coeurs et 50 Go de mémoire vive).
  - Public (hébergement INSEE).
- Utilisation requise pour tous les examens : commencez aujourd'hui !

# Contrôle de version avec Git et GitHub

- Git est un **logiciel de contrôle de version**.
- Enregistre les évolutions d'un fichier de code dans le temps.
- Sauvegarde sur un **dépôt distant** (en ligne), fourni par GitHub (ou autre fournisseur).
- S'utilise facilement grâce à l'interface Git de votre IDE.
- Conçu pour les projets collaboratifs.
- Travail dans des **branches** séparées pour éviter les conflits de version.
- On **tire** (pull) la dernière version du projet au début de sa session.
- On **pousse** (push) ses changements à la fin pour les sauvegarder.
- **Pour ce cours** : uniquement des pull pour récupérer les notebooks.

# Comment créer son service Jupyter-Python

1. Visitez cette URL pour créer votre compte :  
<https://datalab.sspcloud.fr/> (puis cliquez sur **Connexion**).
2. Puis allez à **Mes services** > **Nouveau service** :

The screenshot displays the 'Onyxia - SSPCloud Datalab' web interface. At the top, there's a navigation bar with the logo, project name 'tthivillon personal project', and links for 'Tutoriels', 'AI Chat', 'Contactez nous', and 'Déconnexion'. The main section is titled 'Mes services' and includes a sub-header 'Lancer, visualiser et gérer rapidement vos différents services en cours d'exécution.' Below this, there are buttons for 'Rafraîchir', '+ Nouveau service', 'Supprimer tous', and 'Events'. A terminal window shows the command '\$ helm list --namespace user-tthivillon'. The 'Services en cours' section lists two services: 'BETSAKA\_fires\_202511' and 'Jupyter-python'. The 'Jupyter-python' service is highlighted with a 'Démarré' status and a note 'il y a 3 mois'. To the right, the 'Quotas d'utilisation des ressources' section shows a green checkmark indicating reasonable resource usage. Below that, the 'Enregistrés' section lists the same two services with 'Lancer' buttons.

Onyxia - SSPCloud Datalab

Project: tthivillon personal project

Tutoriels AI Chat Contactez nous Déconnexion

## Mes services

Lancer, visualiser et gérer rapidement vos différents services en cours d'exécution.  
Il est recommandé de supprimer vos services après chaque session de travail.

Rafraîchir + Nouveau service Supprimer tous Events

### Services en cours

```
$ helm list --namespace user-tthivillon
```

| Service              | Statut                 |
|----------------------|------------------------|
| BETSAKA_fires_202511 | Ratudio Suspendu       |
| Jupyter-python       | Démarré: il y a 3 mois |

Ouvrir

### Quotas d'utilisation des ressources

Afficher plus (3)

✓ Votre utilisation actuelle des ressources est raisonnable.

### Enregistrés

Développer (2)

|                      |        |
|----------------------|--------|
| BETSAKA_fires_202511 | Lancer |
| BETSAKA_fires        | Lancer |

# Comment créer son service Jupyter-Python

1. Sélectionnez **Jupyter-Python** dans le catalogue.
2. Faites dérouler le menu du formulaire de paramétrage jusqu'à la section **GIT**.
3. Entrez l'URL du dépôt GitHub du cours sous **Repository url** :  
`https://github.com/thomas-thivillon/python_mageval1.git`
4. Assurez-vous que l'option **Add Git config inside your environment** est activée.
5. Cliquez sur la **disquette** en haut à droite du menu pour sauvegarder votre configuration.
6. Cliquez sur **Lancer**.

# Comment créer son service Jupyter-Python

The screenshot shows the Onyxia web interface for creating a Jupyter-Python service. The header includes the Onyxia logo, the text "SSPCloud Datalab", a project dropdown menu set to "thivillon personal project", and navigation links for "Tutoriels", "AI Chat", "Contactez nous", and "Déconnexion". A terminal bar at the top displays the command: `$ helm install jupyter-python-448015 ide/jupyter-python -f values.yaml`.

The main content area is titled "Jupyter-python" and includes links for "Réinitialiser les configurations", "Copier l'URL de lancement automatique", and "Enregistrer la configuration". A note states: "Le chart Helm jupyter-python appartient au dépôt de charts Helm Interactive services. Il est basé sur l'image Docker [jupyter-python](#)."

Below this, there is a form with two tabs: "Formulaire" (selected) and "Éditeur de texte". The form contains the following fields:

- Nom personnalisé**: `jupyter-python`
- Version**: `2.4.0`

Buttons for "Annuler" and "Lancer" are located to the right of the version field.

The "Formulaire" tab is expanded, showing the "GIT" section with the title "Git user configuration". It includes a toggle switch for "Enabled" (currently on) and a note: "Add git config inside your environment". Below this are input fields for "Name" (User name for git) and "Email" (User email for git).

Further down, there are sections for "Cache" (Duration in seconds of the credentials cache duration, set to 0), "Token" (Personal access token), and "Repository" (Repository url, set to `https://github.com/thomas-thivillon/`).

At the bottom, there is a "Branch" field and a note: "Open the service in the clone folder".

The footer includes the text "Conditions d'utilisation" and a language selector set to "Français", along with the version "Onyxia v10.32.0".

# Les 3 éléments de votre environnement Python

## 1. L'interpréteur Python :

- Le langage de programmation Python lui-même.
- On choisit sa version au moment de paramétrer son service Jupyter-Python dans SSP Cloud.

## 2. Votre environnement Python :

- Ensemble de *packages* que vous installez pour compléter la version de base de Python.
- Stockés sur des plateformes communautaires (ex : PyPI).
- On installe avec la commande `pip install`.

## 3. Votre IDE :

- Permet d'écrire et d'exécuter du code dans une seule fenêtre.
- Les résultats des calculs sont affichés sous les blocs de code.
- Le notebook intègre également des blocs de commentaires (en langage Markdown dans Jupyter).
- Les notebooks peuvent être sauvegardés, modifiés et ré-exécutés à l'infini.
- Outil incontournable pour programmer de façon professionnelle et répliquable.

## Première prise de contact avec un notebook JupyterLab

- Ouvrez le service de Jupyter-Python que vous avez créé précédemment.
- Recherchez le notebook `notebook_decouverte` et ouvrez-le.
- Suivez les instructions.

**Merci de votre attention !**

Contact : [thomas.thivillon@u-bordeaux.fr](mailto:thomas.thivillon@u-bordeaux.fr)